Deduced return type for overriding functions via `auto`

Document #:	DnnnnR0
Date:	2026-06-25
Project:	Programming Language C++
Audience:	SG17 (EWG Incubator) EWG (Evolution Working Group)

Reply-to:	Alex Tsvetanov <alex_tsvetanov_2002@abv.bg>

Contents

1 Abstract

A function that overrides a virtual function must, today, repeat the overridden function's return type — either by spelling it out or by reconstructing it with a trailing-return-type such as `-> decltype(std::declval<Base>().f())`. This paper proposes that a function declared with the bare

placeholder `auto` *and* the `override` specifier deduce its return type to be **exactly the return type of the function it overrides**. Because `override` already guarantees that a matching base function exists, the deduction is unambiguous, requires no function body, and the resulting function is an ordinary virtual function in every other respect.

Before

```
struct A {
  virtual int test() { return 5; }
};

struct B : A {
  // Return type must be restated, either lit
  int test() override { return 6; }

  // ...or reconstructed when A::test's type
  auto test2() -> decltype(std::declval<A>().t
      override { return 6; }
};
```

After

```
struct A {
  virtual int test() { return 5; }
};

struct B : A {
  // Return type is taken from the overridden function.
  auto test() override { return 6; }

  auto test2() override { return 6; }
};
```

2 Introduction

C++ requires an overriding virtual function to repeat the return type of the function it overrides, even though that type is fixed by the language and carries no new information. This paper proposes a small, self-contained extension: a function declared with the bare placeholder `auto` together with the `override` specifier takes its return type directly from the function it overrides. The change is a pure language extension — it makes a presently ill-formed spelling well-formed, alters no existing program, and requires no library, ABI, or object-model changes. The remainder of the paper motivates the feature, discusses the design, and provides proposed wording.

3 Revision history

R0

— Initial revision.

4 Motivation and scope

4.1 The problem

When overriding a virtual function, the override's return type is constrained by the language: it must be *identical* to the base function's return type, or a legal covariant type ([class.virtual]). The programmer therefore gains nothing by restating the return type — it carries no new information, it cannot legally differ (except for covariance), and it must be kept in sync by hand.

This redundancy becomes actively harmful when the base return type is verbose, dependent, or obtained through metaprogramming. The idiomatic way to "inherit" the return type today is:

```
struct B : A {
  auto test() -> decltype(std::declval<A>().test()) override { return 6; }
};
```

That trailing-return-type is pure boilerplate: it names the base class explicitly (`A`), it duplicates the function name (`test`), and it is fragile under refactoring — renaming the base, or moving the function through the hierarchy, silently requires the expression to be updated. Worse, `std::declval`/`decltype` is an advanced idiom that obscures a trivial intent: *"return whatever the base returns."*

4.2 What this paper proposes

Allow a virtual override to be written with the bare placeholder type `auto` and no trailing-return-type:

```
struct B : A {
  auto test() override { return 6; }
};
```

When a function is declared this way, its return type is determined to be **the return type of the unique function it overrides** in a direct or indirect base class. The determination happens at the point of declaration from the override relationship — it does **not** depend on the function body — so the function behaves exactly as if the return type had been written out by hand.

4.3 Why this is safe and small

The proposal deliberately ties the feature to the `override` specifier (see [design.trigger]). The `override` keyword already makes the program ill-formed unless the function overrides a base virtual function ([class.virtual]/4), so by construction there is always exactly one return type to copy. The feature therefore introduces no new failure mode that `override` did not already diagnose: a function marked `override` that fails to override anything is ill-formed today, and remains ill-formed under this proposal.

5 Design discussion

5.1 The feature is triggered only by `override`

Deduction applies only to a function declared with **both** the placeholder type `auto` **and** the `override` specifier. We do *not* extend it to any function that merely happens to match a base virtual signature.

Tying the feature to `override` has three benefits:

— **A return type always exists to copy.** `override` guarantees a matching base function, so the deduction can never silently fail to find a source.
— **Intent is explicit.** The reader sees `override` and knows the function is bound to a base contract; the return type following from that contract is unsurprising.
— **No accidental coupling.** A non-`override` function with a deduced return type continues to deduce from its body, exactly as today. Adding or removing a base virtual function never silently changes the meaning of an unrelated `auto` function.

5.2 The deduced type is the overridden function's type, exactly

The return type is copied verbatim from the overridden function. It is **not** deduced from the override's `return` statements. Two consequences:

— The function body is irrelevant to the return type. A declaration with no definition (e.g. an out-of-line definition) is fully supported:

```
struct B : A {
  auto test() override;     // return type is A::test's type: int
};
int B::test() { return 6; } // out-of-line definition; type already fixed
```

— **Covariance is not synthesized.** Today an override of `Base* f()` may return `Derived*`. Under this proposal, `auto f() override` yields exactly `Base*`. A programmer who wants a covariant return type must still write it explicitly:

```
struct A { virtual A* clone(); };
struct B : A {
  auto  clone() override;   // returns A*  (base type, exactly)
  B*    clone2() override;  // covariant return type, written out (unchanged today)
};
```

This keeps the rule trivially simple — *the override's type is the overridden function's type* — and preserves covariance as an explicit, opt-in act rather than something the compiler infers.

5.3 Ambiguity across multiple base functions

A function may override more than one base function under multiple inheritance. If the overridden functions do not all have the same return type, there is no single type to copy and the program is ill-formed:

```
struct A { virtual int  f(); };
struct C { virtual long f(); };
struct D : A, C {
  auto f() override;  // ill-formed: A::f returns int, C::f returns long
};
```

Note this case is already ill-formed today for a different reason (an override cannot simultaneously satisfy two incompatible base return types), so the proposal does not introduce a new diagnosable situation here; it only specifies which diagnostic applies.

5.4 `decltype(auto)` and other placeholders

This paper addresses only the bare placeholder `auto`. A virtual function declared with `decltype(auto)` or a constrained placeholder (e.g. `std::integral auto`) remains ill-formed, as today. Extending the feature to those spellings is possible future work (see [future]) but is intentionally out of scope to keep the rule minimal.

5.5 Relationship to the existing prohibition on virtual `auto`

Today, a function whose declared return type uses a placeholder type "shall not be declared `virtual`" — a deduced-from-body return type is incompatible with the vtable model because the type must be known to all translation units that see the class definition. This proposal does **not** weaken that rationale: the return type of an `auto … override` function is known from the class definition (it is the base's return type, which is visible wherever the derived class is complete), so every translation unit agrees on the type without seeing the body. The proposal therefore carves a narrow, well-defined exception out of the existing prohibition rather than removing it.

5.6 Interaction with `final`, access, `noexcept`, and qualifiers

`final`, access specifiers, `noexcept`, ref-qualifiers, and cv-qualifiers are orthogonal to the return type and are unaffected. `auto f() const override`, `auto f() && override`, `auto f() noexcept override` all behave as expected: the signature determines which base function is overridden, and that function's return type is copied.

6 Proposed wording

The proposed changes are relative to the C++ working draft [N5046]. Existing standard text is quoted verbatim; inserted text is shown like this and ~~removed text like this~~. Paragraph numbers match N5046 and are to be adjusted editorially. Three paragraphs/tables are affected.

6.1 Change 1 — relax the virtual-function prohibition

In **[dcl.spec.auto.general]** paragraph 17, the placeholder return type is forbidden on *every* virtual function. The current text reads, in its entirety:

17 A function declared with a return type that uses a placeholder type shall not be virtual ([class.virtual]).

Modify it to permit the bare-`auto` + `override` form:

17 A function declared with a return type that uses a placeholder type shall not be virtual ([class.virtual])~~.~~ unless the placeholder type is the `auto` *type-specifier* not followed by a *trailing-return-type* and the function is declared with the `override` *virt-specifier*; in that case the return type of the function is determined from the function it overrides, as specified in [class.virtual]. [Note: The return type of such a function is therefore not deduced from its `return` statements ([dcl.spec.auto.general]). — end note]

Drafting note. No other paragraph of [dcl.spec.auto.general] needs to change: paragraph 8 ("a program that uses a placeholder type in a context not explicitly allowed … is ill-formed") already defers to the per-context rules, and this is the only context that forbade a virtual function.

6.2 Change 2 — specify the return type from the overridden function

In **[class.virtual]**, the override-matching rule (paragraph 2) keys on the *name* and *parameter-type-list* and never on the return type, and paragraph 5 makes `override` ill-formed unless the function overrides something. Add a new paragraph immediately after paragraph 8 (the covariant-return-type rule) to determine the return type in the new case. For reference, paragraph 8 currently begins:

8 The return type of an overriding function shall be either identical to the return type of the overridden function or covariant with the classes of the functions. [...]

Insert after it:

(8.*) If the declared return type of an overriding function `G` uses the `auto` placeholder type ([dcl.spec.auto]), then `G` is declared with the `override` *virt-specifier* ([dcl.spec.auto.general]) and therefore overrides at least one function ([class.virtual]/5). If the functions that `G` overrides do not all have the same return type, the program is ill-formed. Otherwise, the return type of `G` is that common return type. [Note: This return type is identical to the return type of each overridden function and is therefore not covariant ([class.virtual]/8) with it. A covariant return type must be declared explicitly. — end note]

Drafting note. Because paragraph 2's override matching does not consult the return type, the set of overridden functions — and hence "that common return type" — is well-defined before `G`'s return type is known; there is no circularity.

6.3 Change 3 — feature-test macro

In **[cpp.predefined]**, add a row to Table [tab:cpp.predefined.ft] ("Feature-test macros"), in alphabetical order among the other `__cpp_` macros:

Macro name	Value
`__cpp_deduced_virtual_override_return`	`202606L`

Drafting note. The value `202606L` is provisional; the editor assigns the year-and-month (`YYYYMML`) of the meeting at which the feature is adopted. The name uses "virtual" because the feature applies precisely to virtual functions: `override` is permitted only on a function that overrides a base virtual function ([class.virtual]/5), which is therefore itself virtual.

6.4 Summary of edits

#	Stable label	Location	Edit
1	[dcl.spec.auto.general]	para 17	Append an exception permitting `auto` + `override`; point to [class.virtual] for the resulting type.
2	[class.virtual]	new para, after 8	Fix the return type to the overridden function's; make differing return types across multiple overridden functions ill-formed.
3	[cpp.predefined]	Table [tab:cpp.predefined.ft]	Add the `__cpp_deduced_virtual_override_re` feature-test macro.

7 Impact on the Standard

This is a pure language extension. It:

— changes no existing valid program's meaning (code that does not combine `auto` with `override` is unaffected);
— makes well-formed a category of declarations that are ill-formed today (`auto f() override` with no trailing-return-type on a virtual function);
— requires no library changes;
— requires no changes to the ABI or object model — an `auto … override` function has the same type, mangling, and vtable slot it would have had if its return type were written out by hand.

There are no breaking changes and no deprecations.

8 Implementation experience

The feature requires no novel analysis. A conforming compiler already:

1. resolves which base function(s) a member overrides (to validate `override` and covariance), and
2. has the overridden function's return type available at that point.

Implementing the proposal amounts to: when a virtual function's declared return type is the bare `auto` placeholder and `override` is present, set the function's return type to the overridden function's return type (after the existing override resolution) instead of entering body-based deduction. No implementation experience exists yet; the author believes a prototype in a major compiler is straightforward and intends to pursue one.

9 Future directions

The following are intentionally **out of scope** for this paper but are natural follow-ups:

— **`decltype(auto)` overrides.** Could be defined to copy the base type with its value category, though the practical benefit over `auto` is small.
— **Deduced covariant return types.** Allowing `auto` to deduce a covariant type from the body (rather than copying the base type exactly) was considered and rejected for R0 in favor of a simpler rule; it could be revisited.
— **Dropping the `override` requirement.** Permitting deduction for any overriding function was rejected (see [design.trigger]); revisiting it would require a story for silent overrides and accidental coupling.

10 Acknowledgements

To be added.

11 References

12 References

[N5046] 2026-05-12. Working Draft, Programming Languages — C++.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/n5046.pdf